

Continuous exposures with propensity scores

Malcolm Barrett

Stanford University

Warning! Propensity
score weights are
sensitive to positivity
violations for
continuous exposures.

The story so far

Propensity score weighting

- 1 Fit a propensity model predicting exposure x , $x + z$ where z is all covariates
- 2 Calculate weights
- 3 Fit an outcome model estimating the effect of x on y weighted by the propensity score

Continuous exposures

- 1 Use a model like `lm(x ~ z)` for the propensity score model.
- 2 Use `wt_ate()` with `.fitted`; transforms using `dnorm()` to get on probability-like scale.
- 3 Apply the weights to the outcome model as normal!

Alternative: quantile binning

- 1 Bin the continuous exposure into quantiles and use categorical regression like a multinomial model to calculate probabilities.
- 2 Calculate the weights where the propensity score is the probability you fall into the quantile you actually fell into. Same as the binary ATE!
- 3 Same workflow for the outcome model

1. Fit a model for exposure ~ confounders

```
1 model <- lm(  
2   exposure ~ confounder_1 + confounder_2,  
3   data = df  
4 )
```

2. Calculate the weights with `wt_ate()`

```
1 model |>
2   augment(data = df) |>
3   # wt_ate() estimates the SD from the model
4   mutate(wts = wt_ate(
5     .fitted,
6     exposure,
7     exposure_type = "continuous"
8   ))
```

Does change in smoking intensity (**smkintensity82_71**) affect weight gain among lighter smokers?

```
1 nhfs_light_smokers <- nhfs_complete |>  
2   filter(smokeintensity <= 25)
```

1. Fit a model for exposure ~ confounders

```
1 nhefs_model <- lm(  
2   smkintensity82_71 ~ sex + race + age + I(age^2) +  
3   education + smokeintensity + I(smokeintensity^2) +  
4   smokeyrs + I(smokeyrs^2) + exercise + active +  
5   wt71 + I(wt71^2),  
6   data = nhefs_light_smokers  
7 )
```

2. Calculate the weights with `wt_ate()`

```
1 nhfs_wts <- nhfs_model |>
2   augment(data = nhfs_light_smokers) |>
3   mutate(wts = wt_ate(
4     .fitted,
5     smkintensity82_71,
6     exposure_type = "continuous"
7   ))
```

2. Calculate the weights with `wt_ate()`

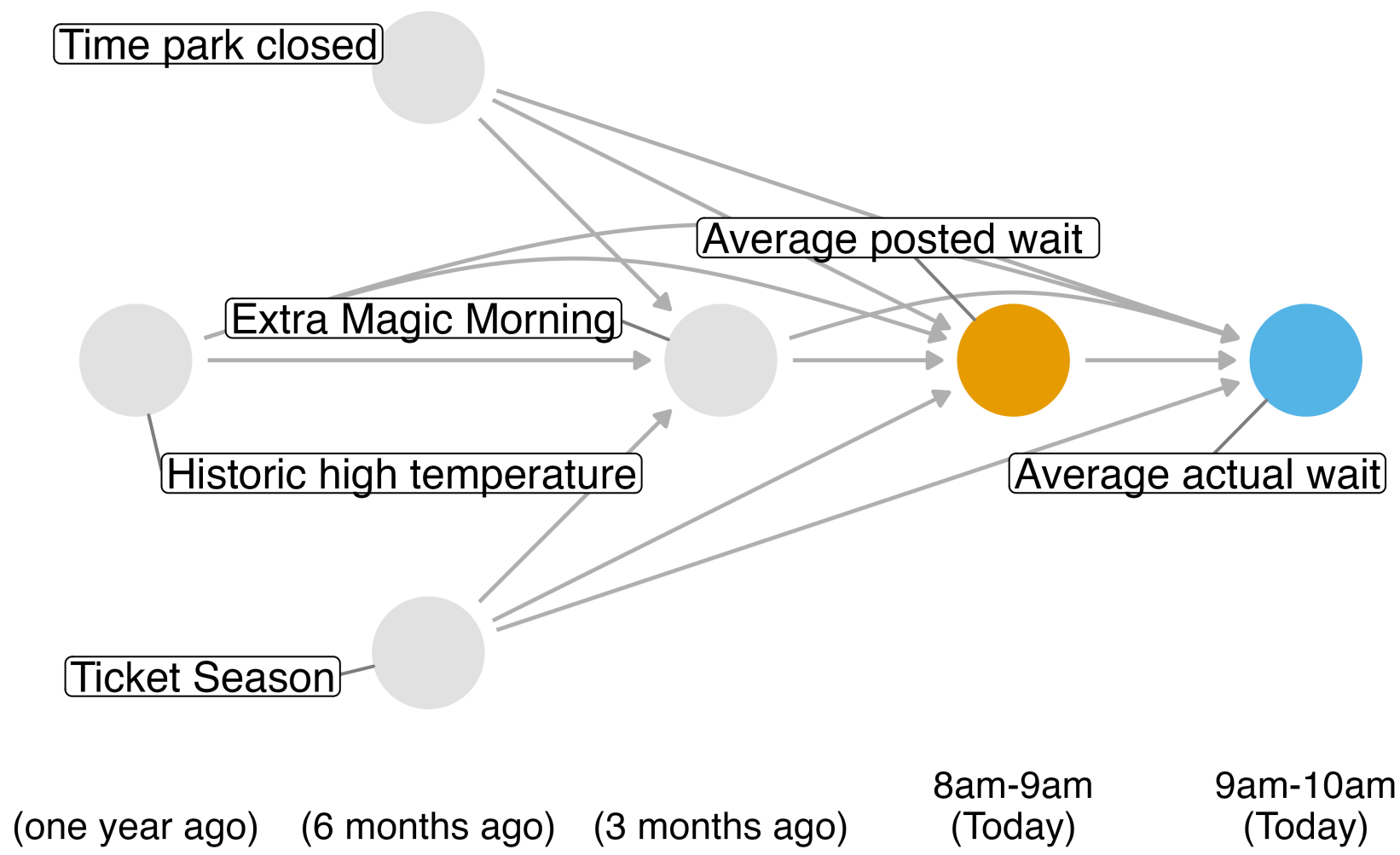
```
1 nhefs_wts
```

```
# A tibble: 1,162 × 74
```

	seqn	qsmk	death	yrdth	modth	dadth	sbp	dbp	sex
	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<fct>
1	235	0	0	NA	NA	NA	123	80	0
2	244	0	0	NA	NA	NA	115	75	1
3	245	0	1	85	2	14	148	78	0
4	252	0	0	NA	NA	NA	118	77	0
5	257	0	0	NA	NA	NA	141	83	1
6	262	0	0	NA	NA	NA	132	69	1
7	266	0	0	NA	NA	NA	100	53	1
8	419	0	1	84	10	13	163	79	0
9	420	0	1	86	10	17	184	106	0
10	434	0	0	NA	NA	NA	127	80	1

```
# ... 1,152 more rows
```

Do *posted* wait times at 8 am affect *actual* wait times at 9 am?



Your Turn 1

Fit a model using `lm()` with `wait_minutes_posted_avg` as the outcome and the confounders identified in the DAG.

Use `augment()` to add model predictions to the data frame

In `wt_ate()`, calculate the weights using `.fitted` and `wait_minutes_posted_avg`

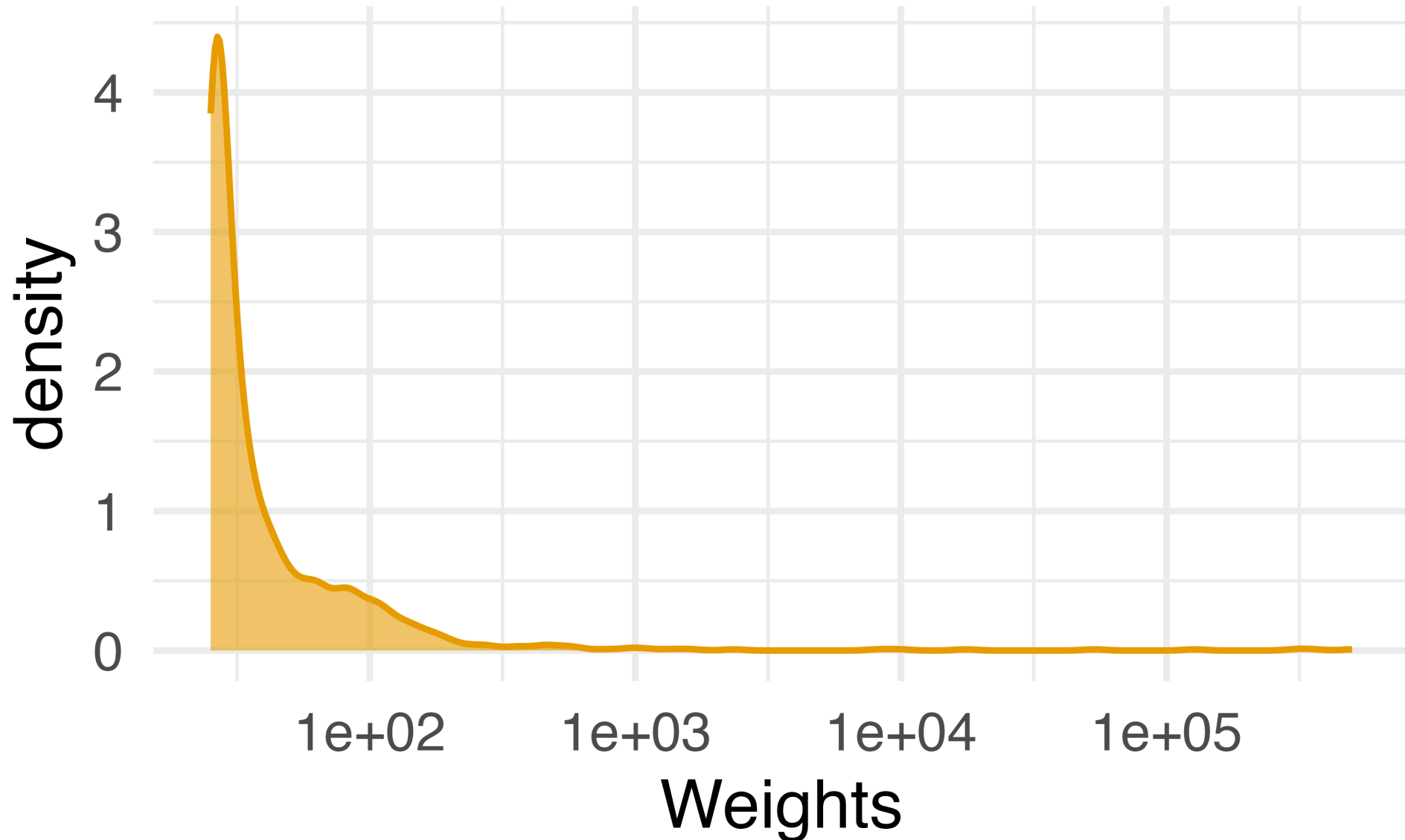
Your Turn 1

```
1 post_time_model <- lm(  
2   wait_minutes_posted_avg ~  
3     park_close + park_extra_magic_morning +  
4     park_temperature_high + park_ticket_season,  
5   data = wait_times  
6 )
```

Your Turn 1

```
1 wait_times_wts <- post_time_model |>
2   augment(data = wait_times) |>
3   mutate(wts = wt_ate(
4     .fitted,
5     wait_minutes_posted_avg,
6     exposure_type = "continuous"
7   ))
```

Stabilizing extreme weights



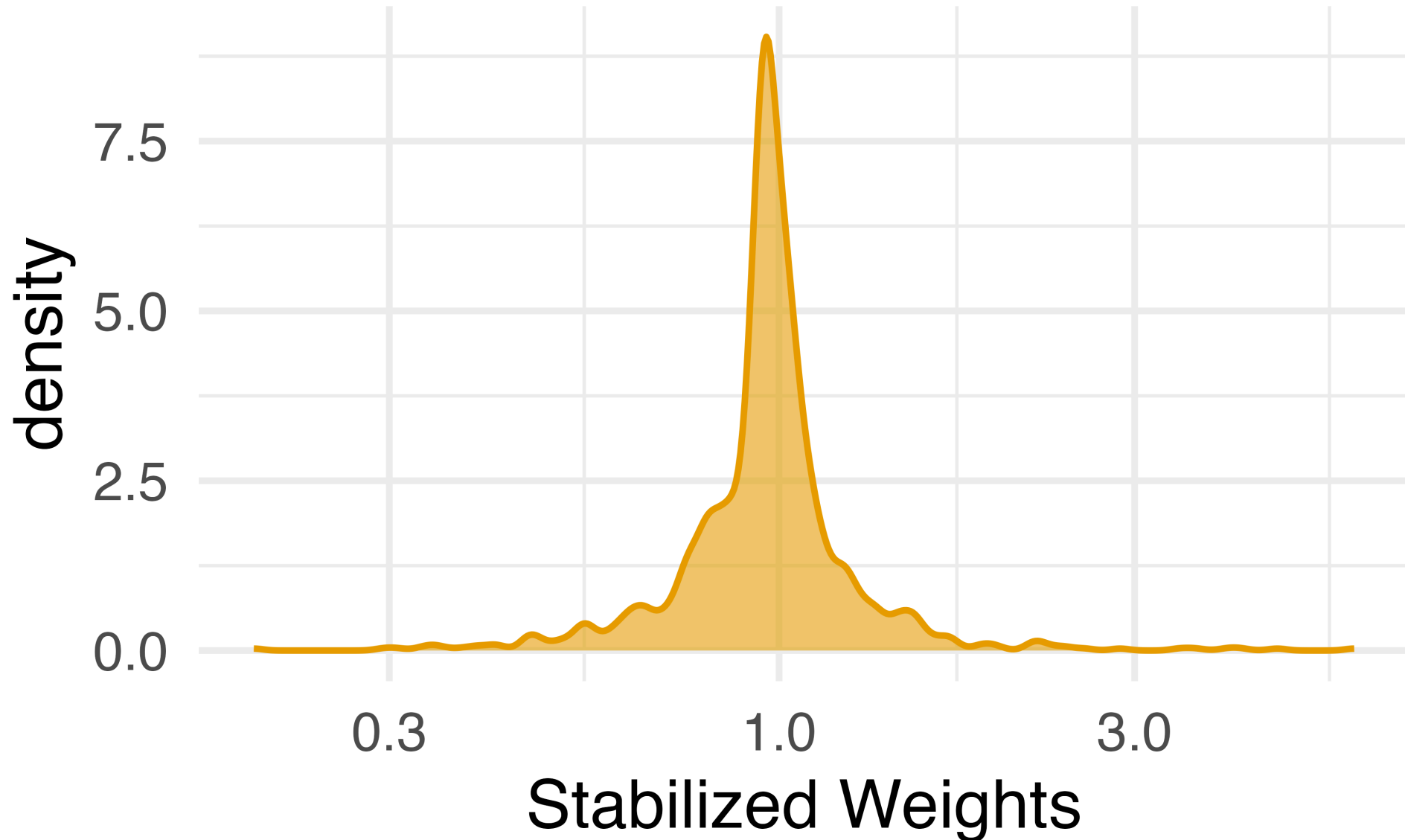
Stabilizing extreme weights

- 1 Fit an intercept-only model (e.g. `lm(x ~ 1)`) or use mean and SD of `x`
- 2 Calculate the marginal density from this model.
- 3 Divide the marginal density by the propensity score density. `wt_ate(..., stabilize = TRUE)` does this all!

Calculate stabilized weights

```
1 nhefs_swts <- nhefs_model |>
2   augment(data = nhefs_light_smokers) |>
3   mutate(swts = wt_ate(
4     .fitted,
5     smkintensity82_71,
6     exposure_type = "continuous",
7     stabilize = TRUE
8   ))
```

Stabilizing extreme weights



Stabilized weights should have a mean close to 1

```
1 mean(nhefs_swts$swts)
```

```
[1] 0.9989975
```

If the mean is far from 1, we may have issues with model misspecification or positivity violations. A check worth running every time.

Your Turn 2

Re-fit the above using stabilized weights

Your Turn 2

```
1 wait_times_swts <- post_time_model |>
2   augment(data = wait_times) |>
3   mutate(swts = wt_ate(
4     .fitted,
5     wait_minutes_posted_avg,
6     exposure_type = "continuous",
7     stabilize = TRUE
8   ))
```

Fitting the outcome model

- 1 Use the stabilized weights in the outcome model. Nothing new here!
- 2 Use `ipw()` for standard errors that account for the weights

```
1 lm(  
2   wt82_71 ~ smkintensity82_71,  
3   weights = swts,  
4   data = nhefs_swts  
5 ) |>  
6 tidy() |>  
7 filter(term == "smkintensity82_71") |>  
8 mutate(estimate = estimate * -10)
```

A tibble: 1 × 5

	term	estimate	std.error	statistic	p.value
	<chr>	<dbl>	<dbl>	<dbl>	<dbl>
1	smkintensity82_71	0.965	0.0206	-4.68	0.000000323

Fit the outcome model

```
1 nhefs_ipw_model <- lm(  
2   wt82_71 ~ smkintensity82_71,  
3   weights = swts,  
4   data = nhefs_swts  
5 )
```

Estimate the effect with `ipw()`

```
1 nhefs_ipw_fit <- ipw(nhefs_model, nhefs_ipw_model)
```

`ipw()` uses a sandwich estimator that also accounts for the uncertainty in the propensity score model

Estimate the effect with `ipw()`

```
1 nhfs_ipw_fit
```

Inverse Probability Weight Estimator

Estimand: ATE

Effects: marginal (population-averaged)

Weight Estimator:

```
Call: lm(formula = smkintensity82_71 ~ sex + race + age + I(age^2) +
  education + smokeintensity + I(smokeintensity^2) + smokeyrs +
  I(smokeyrs^2) + exercise + active + wt71 + I(wt71^2), data = nhfs_light_smokers)
```

Outcome Model:

```
Call: lm(formula = wt82_71 ~ smkintensity82_71, data = nhfs_swts,
  weights = swts)
```

Marginal estimates:

	estimate	std.err	z	ci.lower	ci.upper	conf.level	p.value
slope	-0.096497	0.036518	-2.6424	-0.16807	-0.024923	0.95	0.008231 **

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Your Turn 3

Estimate the relationship between posted wait times and actual wait times using the stabilized weights we just created.

Fit the same outcome model again, saving it to an object

Use `ipw()` to estimate the effect with standard errors that account for the weights, then `as_conditional()` to switch readings

Your Turn 3

```
1 lm(  
2   wait_minutes_actual_avg ~ wait_minutes_posted_avg,  
3   weights = swts,  
4   data = wait_times_swts  
5 ) |>  
6 tidy() |>  
7 filter(term == "wait_minutes_posted_avg") |>  
8 mutate(estimate = estimate * 10)
```

A tibble: 1 × 5

	term	estimate	std.error	statistic	p.value
	<chr>	<dbl>	<dbl>	<dbl>	<dbl>
1	wait_minutes_posted_...	-2.91	0.0725	-4.01	1.30e-4

Your Turn 3

```
1 ipw_model <- lm(  
2   wait_minutes_actual_avg ~ wait_minutes_posted_avg,  
3   weights = swts,  
4   data = wait_times_swts  
5 )  
6  
7 ipw_fit <- ipw(post_time_model, ipw_model)
```

ipw() reports the effect of one more minute, so multiply by 10 to compare

Your Turn 3

```
1 ipw_fit
```

Inverse Probability Weight Estimator

Estimand: ATE

Effects: marginal (population-averaged)

Weight Estimator:

```
Call: lm(formula = wait_minutes_posted_avg ~ park_close + park_extra_magic_morning +
  park_temperature_high + park_ticket_season, data = wait_times)
```

Outcome Model:

```
Call: lm(formula = wait_minutes_actual_avg ~ wait_minutes_posted_avg,
  data = wait_times_swts, weights = swts)
```

Marginal estimates:

	estimate	std.err	z	ci.lower	ci.upper	conf.level	p.value
slope	-0.29116	0.10991	-2.649	-0.50659	-0.075731	0.95	0.008074 **

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Your Turn 3

```
1 as_conditional(ipw_fit) |>  
2   tidy(conf.int = TRUE)
```

```
# A tibble: 2 × 7  
  term          estimate std.error statistic  p.value conf.low  
  <chr>          <dbl>    <dbl>    <dbl>    <dbl>    <dbl>  
1 (Intercept)    44.9      4.51      9.95 2.41e-23    36.0  
2 wait_minut... -0.291     0.110    -2.65 8.07e- 3   -0.507  
# i 1 more variable: conf.high <dbl>
```

The slope matches the marginal reading here because the outcome model is linear in the exposure with an identity link

Causal dose-response functions

- 1 Calculate the stabilized weights
- 2 Fit the outcome model with a flexible term for the exposure
- 3 Use `ipw()` for standard errors that account for the weights
- 4 Predict the outcome across the range of the exposure

1. Calculate the stabilized weights

nhfs_swts already has the stabilized weights

2. Fit a flexible outcome model

```
1 nhefs_msm <- lm(  
2   wt82_71 ~ splines::ns(smkintensity82_71, df = 3),  
3   weights = swts,  
4   data = nhefs_swts  
5 )
```

3. Estimate the effects with `ipw()`

```
1 nhefs_ipw <- ipw(  
2   nhefs_model,  
3   nhefs_msm,  
4   .data = nhefs_swts  
5 )
```

The standard errors account for the estimated weights.

4. Predict across the exposure range

```
1 dose_response <- nhefs_ipw |>
2   as_conditional() |>
3   # marginalesffects needs the data to build the grid
4   set_modeldata(nhefs_swts) |>
5   predictions(
6     newdata = datagrid(smkintensity82_71 = -25:25)
7   )
8
9 dose_response
```

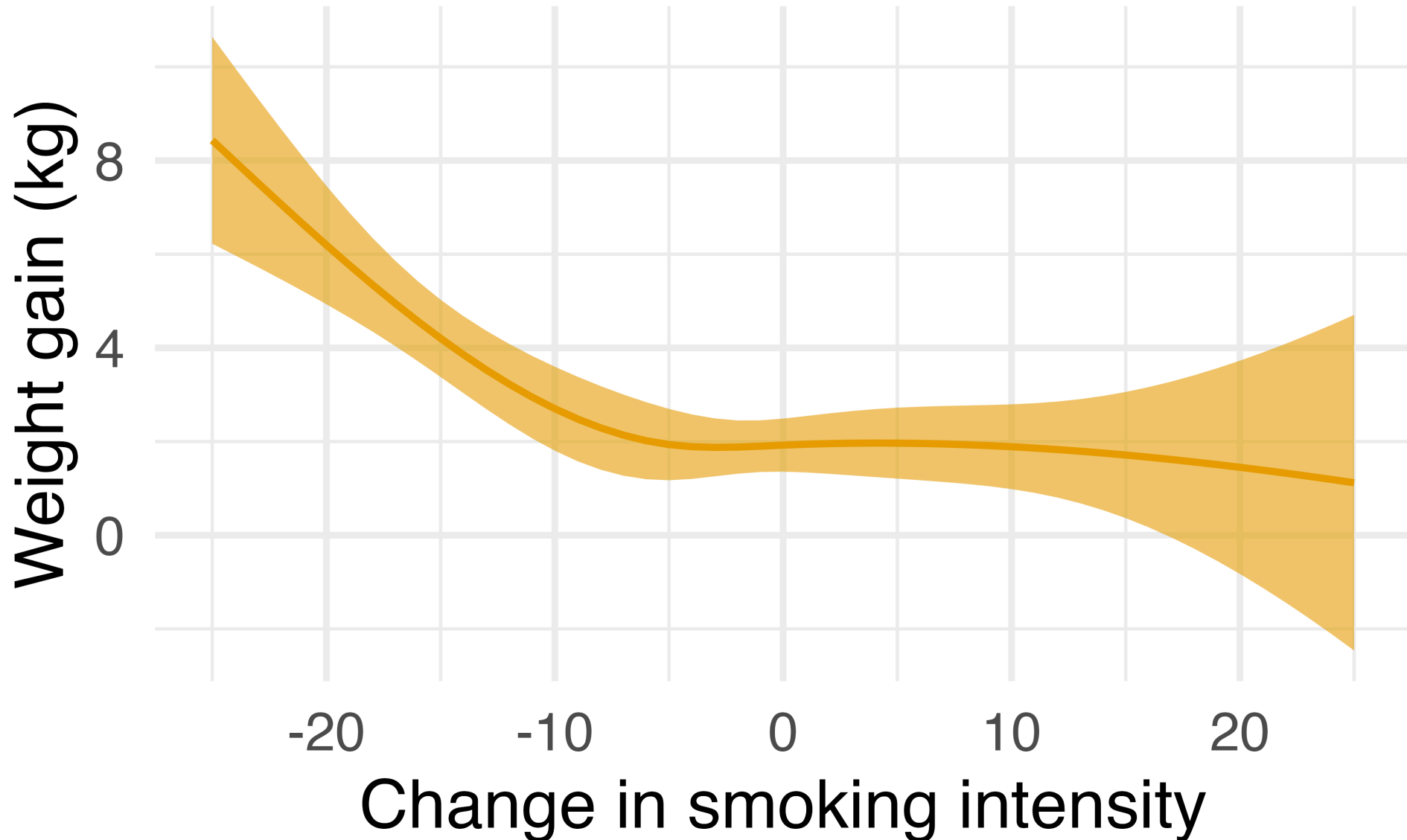
The predictions along the exposure grid come from the outcome model itself, so we switch to the conditional reading first

4. Predict across the exposure range

smkintensity82_71	Estimate	Std. Error	z	Pr(> z)	S	2.5 %	97.5 %
-25	8.43	1.126	7.485	<0.001	43.7	6.22	10.64
-24	7.98	1.021	7.815	<0.001	47.4	5.98	9.98
-23	7.52	0.918	8.194	<0.001	51.8	5.72	9.32
-22	7.07	0.820	8.626	<0.001	57.1	5.47	8.68
-21	6.63	0.728	9.103	<0.001	63.3	5.20	8.06
--- 41 rows omitted. See ?print.marginaleffects ---							
21	1.39	1.282	1.084	0.278	1.8	-1.12	3.90
22	1.33	1.409	0.941	0.347	1.5	-1.44	4.09
23	1.26	1.542	0.817	0.414	1.3	-1.76	4.28
24	1.19	1.682	0.708	0.479	1.1	-2.11	4.49
25	1.12	1.828	0.613	0.540	0.9	-2.46	4.70

Type: response

The causal dose-response function



Diagnosing issues

- 1 Extreme weights even after stabilization
- 2 Bootstrap: non-normal distribution
- 3 Bootstrap: estimate different from original model

More info

Causal Inference in R: g-computation for continuous exposures

<https://github.com/LucyMcGowan/writing-positivity-continuous-ps>